

BattleOps: Week 3

Theme: Kubernetes Security, Platform Hardening & Real-World Incident Prevention

Objective: Apply Kubernetes security concepts in real production-like situations, debug misconfigurations, enforce policies, and secure workloads end-to-end.

TUESDAY: Enterprise RBAC, Namespace Isolation & Multi-Tenant Hardening

ADVANCED CONCEPTS

- Designing RBAC for real teams
- Locking down “break-glass” access
- How teams accidentally gain cluster-admin
- Detecting dangerous RoleBindings
- Namespace boundary enforcement

KEY OUTCOMES

- You can audit RBAC & detect misconfigurations
 - You can build least-privilege roles
 - You can isolate teams with a proper namespace strategy
-

WHAT TO PRACTICE

Lab 1: RBAC Audit

Run:

- `kubectl get clusterrolebindings`
- `kubectl get rolebindings -A`

Identify:

- Who has cluster-admin?
- Which bindings should not exist?

Lab 2: Build a Least-Privilege Role

Only allow: `get/list/watch`

on pods + deployments.

Lab 3: Enforce Namespace Isolation

Create:

- TeamA namespace with its own SA
- TeamB namespace with blocked cross-access

Test with: `kubectl auth can-i <verb> <resource> --as <serviceaccount>`

WHERE TO STUDY

- RBAC Authorization (official docs)
- CNCF “Multi-tenant Kubernetes” whitepaper

INFRATHRONE BATTLEOPS NOTES

- 70% of production breaches happen because RBAC was too open.
- Never give developers access to kube-system.
- Each team needs its own namespace + SA + role set.